Machine Learning Assignment 1

Topics: Optimization and Linear Regression

Total Marks: 10

Q1. Perform linear regression using the closed-form solution (normal equation):

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

where $\mathbf{X}$ is the $N \times (D+1)$ matrix of input features, N is the number of training examples, D is the dimensionality of the feature vector, and $\mathbf{y}$ is the target vector of size $N \times 1$ such that $y^{(i)}$ corresponds to the target value of sample $\mathbf{x}^{(i)}$. The weight vector $\mathbf{w}$ is of size $(D+1) \times 1$. Note that the first column of $\mathbf{X}$ consists of all ones corresponding to the intercept term.

1. On the datasets provided.
2. For the following $\mathbf{X}$ and $\mathbf{y}$, use both the normal equation and the `scikit-learn` implementation to learn a linear regression model:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & 8 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}$$

You may observe that one of the matrices involved in the normal equation is non-invertible. Explain why this matrix is non-invertible and why the `scikit-learn` implementation is still able to correctly solve this regression problem.

Q2. Implement batch, mini-batch, and stochastic gradient descent methods for optimizing the least squares error metric $J(\mathbf{w})$:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - h_{\mathbf{w}}(x^{(i)}) \right)^2$$

where the hypothesis function is defined as

$$h_{\mathbf{w}}(x) = \mathbf{w}^\top \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n.$$

Choose an appropriate learning rate and a stopping criterion based on the change in the value of $J(\mathbf{w})$. You may initialize the parameters as $\mathbf{w} = \mathbf{0}$ (a vector of all zeros). Report the chosen learning rate and the final weight vector obtained by each optimization method.

- Plot the data on a two-dimensional graph and overlay the hypothesis function $h_{\mathbf{w}}(x)$ learned by your algorithm.
- Draw a three-dimensional mesh plot showing the error function $J(\mathbf{w})$ on the z-axis and the parameters $\mathbf{w}$ on the x–y plane. Display the error value using the current set of parameters at each iteration of gradient descent. Include a time gap of 0.2 seconds between iterations so that the change in the function value can be observed clearly.
- Create a Matplotlib animation consisting of two subplots arranged side by side:
  - The first subplot should show the contour plot of $J(w_0, w_1)$.
  - The second subplot should show the linear regression fit on the data.

  The different frames in the animation should correspond to successive iterations of gradient descent used to learn w_0 and w_1. For each iteration, plot the current values of w_0 and w_1 on the contour plot along with an arrow pointing to the next values obtained using the gradient update rule. Correspondingly, draw the line $y = w_0 + w_1 x$ on the second subplot showing the scatter plot of the data.

  The overall title of the animation should indicate the iteration number and the residual sum of squares.

Q3. Let the true function be defined as

$$f(x) = x + 2\sin(0.5x).$$

1. **Data Generation:** Generate 50 data points for $x \in [-10, 10]$ by adding noise drawn from a Gaussian distribution with mean 0 and variance 0.5 to the true function.

2. **Data Visualization and Model Selection:** Plot the generated data points. What could be the probable hypothesis to fit the function? Is fitting a straight line through the data points sufficient? For polynomial regression, vary the degree from 0 to 10.

   - Plot the data points, the true function, and the learned curve for each polynomial degree.
   - Comment on the optimal model capacity for the given data. Identify the polynomial degrees that lead to underfitting and overfitting.

3. **Optimal Model Capacity:** Using the learned weight vector $\mathbf{w}$, predict the labels on a test dataset and compute the squared error. Plot the training error and generalization error as a function of the model capacity (polynomial degree).

4. **Bias and Variance Analysis:** For different values of model capacity (polynomial degree), compute the bias^2 and variance. For this computation, consider $L = 100$ bootstrapped datasets sampled from the original function, each containing 50 training data points. Use a single test set with $N = 200$ data pairs to evaluate the models learned on all L datasets.

   The bias^2 and variance are defined as:

$$\text{bias}^2 = \frac{1}{N}\sum_{n=1}^{N}\left(\bar{y}(x_n) - y(x_n)\right)^2$$

$$\text{variance} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{L}\sum_{l=1}^{L}\left(y^{(l)}(x_n) - \bar{y}(x_n)\right)^2$$

   where

$$\bar{y}(x) = \frac{1}{L}\sum_{l=1}^{L} y^{(l)}(x).$$

   Plot the bias^2, variance, training error, and generalization error as functions of model capacity and explain your observations. What is the optimal model capacity?

5. **Addressing Overfitting and Regularization:**

   - **Using More Data:** For the case exhibiting maximum overfitting, sample additional training data points (e.g., 100 points). Plot the true function, the originally learned curve, and the curve learned using more data. Comment on your observations.
   - **Lasso Regression:** Perform Lasso regression by minimizing the following cost function, varying λ from 0 to 5 in steps of 0.5:

$$J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - (w_0 + \mathbf{w}^\top \boldsymbol{\phi}_n(x_i))\right)^2 + \lambda\|\mathbf{w}\|_1$$

     - Plot training and test errors for different values of λ (or $\ln\lambda$).
     - Identify the most suitable value of λ.
     - Examine whether the weight vector $\mathbf{w}$ is sparse and plot the number of non-zero components of $\mathbf{w}$ versus λ.
     - Plot the true function and the curves learned with and without regularization, and compare the results.
     - Plot the L_1 norm of $\mathbf{w}$ as a function of λ.

- **Ridge Regression:** Perform Ridge regression by minimizing the following cost function, varying λ from 0 to 5 in steps of 0.5:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - (w_0 + \mathbf{w}^\top \boldsymbol{\phi}_n(x_i)) \right)^2 + \lambda \|\mathbf{w}\|_2^2$$

  - Plot training and test errors for different values of λ (or $\ln \lambda$).
  - Identify the most suitable value of λ.
  - Plot the L_2 norm of $\mathbf{w}$ versus λ.
  - Plot the true function and the curves learned with and without regularization, and compare the results.
- Compare Lasso and Ridge regression and comment on whether Lasso produces a sparser solution than Ridge regression.